CSE 209 — Computer Architecture

Chapter 3 · Arithmetic for Computers (short slides)

Lesson notes · Exam approach · TF practice with solutions · Quick notes (Tushar Sir's portion)

How to Read These Notes

EXAM FOCUS	Exam importance, where it appears in the TF collection
MEMORIZE	Rules, tables and formulas to memorize
TRAP	Common mistakes and exam traps
EXAM-READY ANSWER	Model wording you can write in the exam
NOTE	Extra explanation / clarification
Grey box, monospace	Code, worked tables, derivations and hand-drawn sketches
Centered grey box	Formula to remember / final numerical result
Purple heading / ✓ green	Practice questions / their solutions

Contents

Exam Map — What the TF Papers Ask — arithmetic questions and how much they matter
Lesson 19 — Integer Addition, Subtraction and Overflow — overflow rules, the O/SA/SB/SR table
Lesson 20 — Multiplication Hardware — first and refined multiplier, flowcharts, step-by-step traces
Lesson 21 — IEEE 754 Floating-Point Representation — format, bias, ranges, special values, custom formats
Lesson 22 — Floating-Point Addition and the FP Adder — 4 steps, flowchart, over/underflow, traced examples
Lesson 23 — Beyond the Short Slides (textbook insurance) — division, signed multiplication, MIPS FP code, guard/round/sticky, subword parallelism
Quick Notes — Arithmetic — merged revision notes + key diagrams

> **NOTE** Slide numbers are PDF pages of Chapter_03_Arithmetic-shortslide.pdf (Tushar Sir, 20 slides; from PDF page 17 on the printed slide number is 2 higher). Lesson 23 covers textbook topics that are NOT on the short slides but are asked repeatedly (division even in 2022-23 CSE 209); those questions sit in the tracker's Outside Notes tab. All traces and encodings were generated and checked by _build/traces.py and verify.py.

Exam Map — What the TF Papers Ask

Every Tushar-Sir-part question that this deck answers (★ = 2019-20 or later):

Session (paper)	Q	What is asked	Marks	Lesson
2023-24 (CSE 209) ★	7(a)	Multiplication hardware: long-multiplication and parallel add/shift versions (32-bit); simulate 1000 × 1011 on each	20	L20
2023-24 (CSE 209) ★	7(b)	Purpose of a positive bias in IEEE 754; smallest and largest values in single and double precision	15	L21
2022-23 (CSE 209) ★	1(a)	Flowchart of floating-point addition; conditions for overflow, underflow, NaN and infinity	10	L22
2022-23 (CSE 209) ★	4	Hardware block diagram + flowchart of the sequential and the refined multiplication algorithm	15	L20
2022-23 (CSE 305) ★	8(b)	32-bit FP with 19-bit fraction: precision, smallest/largest absolute values, smallest denormal	8	L21
2022-23 (CSE 305) ★	8(c)	f1 < f2 (IEEE single): treated as unsigned, is i1 < i2? As signed, is j1 < j2?	20	L21
2021-22 (CSE 305) ★	6(c)	Block diagram of 32-bit combined multiply/divide hardware with a 32-bit ALU; functions of components	10	L20
2021-22 (CSE 305) ★	8(b)	Largest n such that every integer 1…n is exactly representable in IEEE single precision	14	L21
2020-21 (CSE 305) ★	13(a)	32-bit FP with 24-bit fraction: why bias, value of bias, smallest/largest absolute values, smallest denormal	16	L21
2019-20 (CSE 305) ★	4(a)	16-bit FP, largest 1.11…×2^63, smallest 1.00…×2^−62: format; $-110.11_2 \times 2^1$; smallest denormal	15	L21
2019-20 (CSE 305) ★	4(b)	Why the first multiplier hardware is inefficient; changes; draw the efficient hardware	12	L20
2018-19 (CSE 305)	5(a)	Hardware + flowchart of the sequential multiplication algorithm; flowchart of the refined version	12	L20
2018-19 (CSE 305)	5(b)	16-bit FP with 4-bit exponent: represent −1.25 and 0.375	8	L21
2018-19 (CSE 305)	5(c)	Decimal value of IEEE single 0xDF807500	5	L21
2018-19 (CSE 305)	7(c)	Use of bias in FP representation; encoding of infinity	5	L21
2018-19 (CSE 305)	8(b)	Steps of FP addition with an example; overflow/underflow conditions	10	L22
2017-18 (CSE 305)	5(b)	16-bit half precision (4-bit exponent): convert 0.75 and 1.25 and add step by step	15	L22
2017-18 (CSE 305)	5(c)	Decimal value of IEEE single 0x7F800005	5	L21
2016-17 (CSE 305)	7(b)	Simulate FP addition −0.25 + 0.625 on the FP adder hardware (fill the tables)	15	L22
2016-17 (CSE 305)	8(a)	Overflow conditions in the O/SA/SB/SR table for add/subtract; Boolean function for V	10	L19
2016-17 (CSE 305)	8(c)	Simulate unsigned multiplication 0110 × 0110 on the refined (shift-right product) hardware	15	L20
2015-16 (CSE 305)	6(a)	Steps of floating-point addition with an example	12	L22
2015-16 (CSE 305)	7(c)	Use of bias in FP representation; encoding of infinity	10	L21
2014-15 (CSE 305)	8(a)	Overflow conditions in the O/SA/SB/SR table; Boolean function for V	12	L19
2014-15 (CSE 305)	8(b)	Flowchart of the sequential multiplication; draw the register-optimized refined hardware	13	L20
2013-14 (CSE 305)	3(a)	Steps for adding two binary floating-point numbers	10	L22
2013-14 (CSE 305)	3(b)	IEEE 754 single-precision representation of −0.75	10	L21
2013-14 (CSE 305)	3(c)	Block diagram + flowchart for 32-bit unsigned multiplication with no separate multiplier register	15	L20

Session (paper)	Q	What is asked	Marks	Lesson
2012-13 (CSE 305)	8(a)	Purpose of biased notation for FP exponents	5	L21
2011-12 (CSE 305)	5(a)	Flow diagram of FP addition; overflow/underflow conditions	10	L22
2011-12 (CSE 305)	6(a)	First and refined multiplication hardware; improvement of the refined version	15	L20
2022-23 (CSE 209) ★	1(b)	6-bit ALU division: dividend 1011 1100 1111, divisor 110101 — registers at each step [Outside Notes]	10	L23
2022-23 (CSE 305) ★	8(a)	MIPS FP code: float a,b,c; double w,x,y,z; if (a+b>c) x = (y*z)/w; [Outside Notes]	7	L23
2021-22 (CSE 305) ★	5(c)	How can subword parallelism accelerate matrix multiplication / image processing? [Outside Notes]	10	L23
2020-21 (CSE 305) ★	10(c)	Briefly describe how to perform a signed multiplication [Outside Notes]	5	L23
2020-21 (CSE 305) ★	12(c)	Importance of guard digit, round digit and sticky bit when rounding [Outside Notes]	9	L23
2020-21 (CSE 305) ★	13(c)	MIPS FP code: float a,b,c; double w,x,y,z; if (a+b>c) x = (y*z)/w; [Outside Notes]	7	L23
2019-20 (CSE 305) ★	4(c)	MIPS FP code (double): if (a>b) c = c−(c*e)/d; else c = c+d; [Outside Notes]	8	L23
2018-19 (CSE 305)	6(b)	4-bit ALU division: dividend 11001001, divisor 1011 — registers at each step [Outside Notes]	15	L23
2017-18 (CSE 305)	8(a)	4-bit ALU division: dividend 11010101, divisor 1011 — registers at every step [Outside Notes]	15	L23
2017-18 (CSE 305)	8(b)	MIPS FP code for float convert(float cels): ferh = 9.0*cels/5.0 + 32.0 [Outside Notes]	10	L23
2012-13 (CSE 305)	8(b)	Flowchart of division for the given (refined) division hardware [Outside Notes]	12	L23

EXAM FOCUS Arithmetic is asked in every paper (155 marks since 2019-20 from the slides, plus the Lesson 23 rows). Three repeating types: (1) multiplication hardware — draw both versions, flowchart, trace (9 papers); (2) floating-point representation — custom 16/32-bit formats, bias, smallest/largest, hex ↔ decimal (13 rows); (3) FP addition steps / flowchart / traced example (7 papers). Division and MIPS FP code are not on the short slides but appear 4 + 4 times — do Lesson 23 too.

Lesson 19 — Integer Addition, Subtraction and Overflow

Slides 2–4 (§3.1–3.2)

1. Addition (sl. 3)

```
Example 7 + 6 (carries in brackets):
      (0)(0)(1)(1)(0)        carries
  …    0  0  1  1  1         7
  …    0  0  1  1  0         6
  …    0  1  1  0  1        13
```

2. Subtraction (sl. 4)

Subtraction adds the negation of the second operand: 7 − 6 = 7 + (−6) → 0000…0111 + 1111…1010 = 0000…0001.

3. Overflow Rules (sl. 3–4) ★

Operation	Operand A	Operand B	Overflow if result sign is
A + B	≥ 0	≥ 0	1 (negative)
A + B	< 0	< 0	0 (non-negative)
A + B	opposite signs		never
A − B	≥ 0	< 0	1
A − B	< 0	≥ 0	0
A − B	same signs		never

Overflow means the true result is outside the 2's-complement range $-2^{n-1} \ldots 2^{n-1} - 1$. In hardware it is detected as $V = C_n \oplus C_{n+1}$ (Lesson 9).

Exam Approach

1. Turn subtraction into addition of the negated B: A − B overflows exactly when A + (−B) would.
2. Fill the sign table row by row; write V as a sum of minterms (no simplification needed).

Practice — Lesson 19

(a) Actual — 2016-17 8(a) = 2014-15 8(a). (b) Constructed.

(a) The table lists O (0 = A + B, 1 = A − B), S_A, S_B (sign bits) and S_R (sign of the result). Identify the overflow conditions and write V as a Boolean function. (10/12)

(b) 4-bit: does +5 + +4, −6 − +3, +5 − −2 overflow?

✓ Solution

O	S_A	S_B	Effective operation	Overflow possible?	V
0	0	0	(+) + (+)	yes, if $S_R = 1$	S_R
0	0	1	(+) + (−)	no	0
0	1	0	(−) + (+)	no	0
0	1	1	(−) + (−)	yes, if $S_R = 0$	S_R'
1	0	0	(+) − (+)	no	0
1	0	1	(+) − (−) = (+) + (+)	yes, if $S_R = 1$	S_R

O	S_A	S_B	Effective operation	Overflow possible?	V
1	1	0	(−) − (+) = (−) + (−)	yes, if $S_R = 0$	S_R'
1	1	1	(−) − (−)	no	0

$$V = O' \cdot S_A' \cdot S_B' \cdot S_R + O' \cdot S_A \cdot S_B \cdot S_R' + O \cdot S_A' \cdot S_B \cdot S_R + O \cdot S_A \cdot S_B' \cdot S_R'$$

NOTE The printed definition says 'V = 0: overflow occurred, 1: no overflow possible' — that is clearly swapped (V = 1 must mean overflow). State the convention you use; the function above has V = 1 for overflow.

(b) +5 + 4 = 0101 + 0100 = 1001: positive + positive gives sign 1 → overflow (9 > 7). −6 − 3 = 1010 + 1101 = (1)0111: negative + negative gives sign 0 → overflow (−9 < −8). +5 − (−2) = 0101 + 0010 = 0111 = +7 → no overflow.

Lesson 20 — Multiplication Hardware

Slides 5–8 (§3.3)

1. Long Multiplication (sl. 5)

```
      1000      multiplicand (8)
    × 1001      multiplier   (9)
      1000
     0000
    0000
   1000
   1001000      product (72) — length of product = sum of operand lengths
```

2. First Version of the Hardware (sl. 5–6) ★

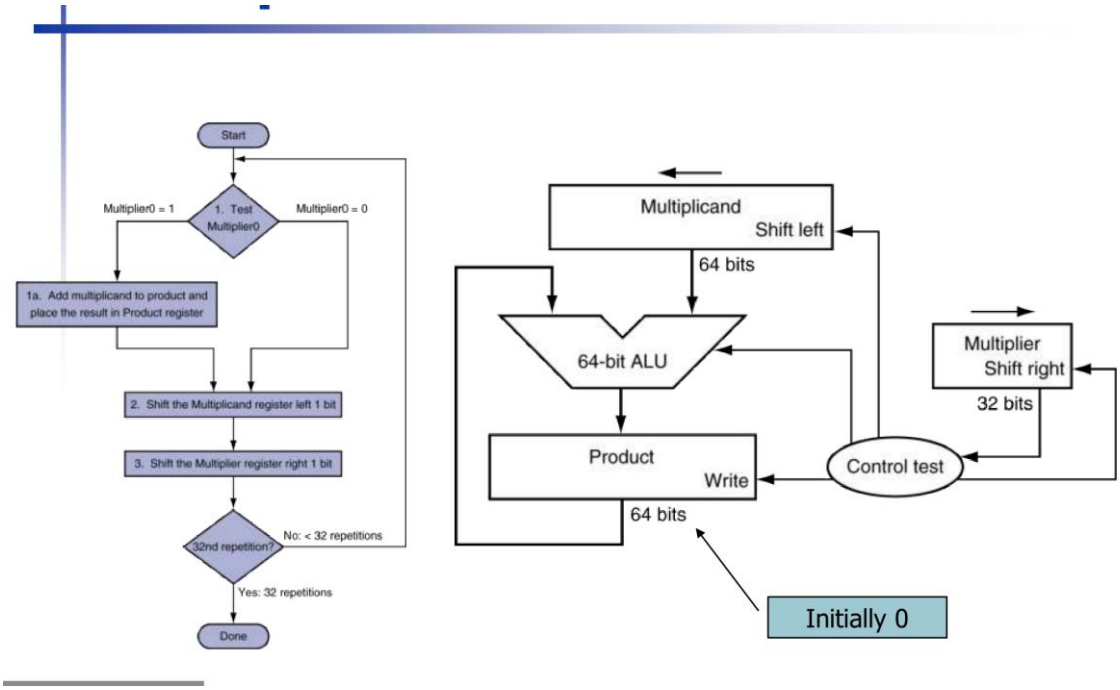


Figure 20.1 — First multiplication algorithm (flowchart) and hardware (sl. 6)

- 64-bit **Multiplicand** register, initially the 32-bit multiplicand in the right half — shifts **left** each step.
- 32-bit **Multiplier** register — shifts **right**; its bit 0 is tested.
- 64-bit **ALU** and 64-bit **Product** register (initially 0).
- Control test repeats 32 times: 1. if Multiplier0 = 1 add Multiplicand to Product; 2. shift Multiplicand left 1; 3. shift Multiplier right 1.

```
Start → Test Multiplier0 ──1──► 1a. Product = Product + Multiplicand ─┐
                          └──0──────────────────────────────────────────┤
        2. Shift the Multiplicand register left 1 bit ◄────────────────┘
        3. Shift the Multiplier register right 1 bit
        32nd repetition? ── No (< 32) → back to Test   ── Yes → Done
```

3. Optimized (Refined) Multiplier (sl. 7) ★

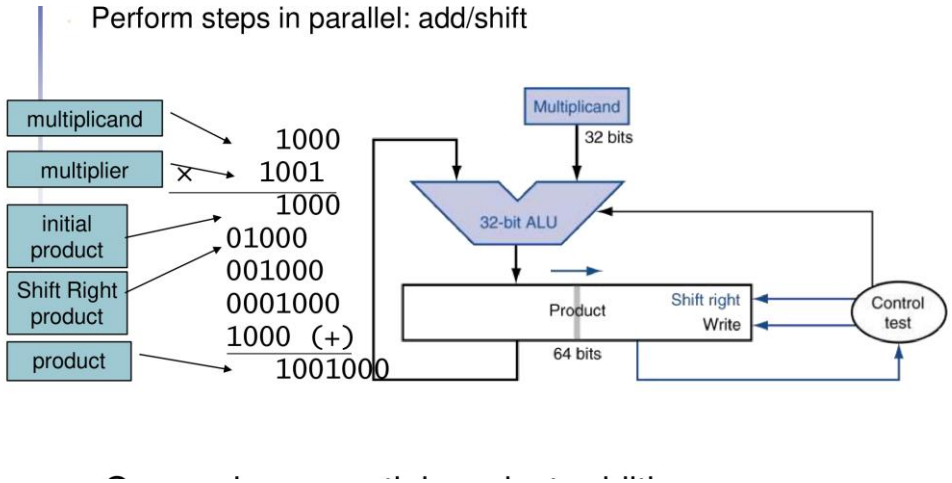


Figure 20.2 — Optimized multiplier: 32-bit multiplicand and ALU, 64-bit product shifting right (sl. 7)

Observations that make it efficient: half of the 64-bit multiplicand register was always 0 and only 32 bits are added each step; the product shifts right as the multiplier shifts right; the product's empty right half can hold the multiplier. So:

- Multiplicand and ALU become **32 bits**; the separate Multiplier register is **removed** — the multiplier is loaded into the right half of the 64-bit Product register.
- Each step: if Product0 = 1, add Multiplicand to the left half of Product (the carry is kept); then shift the whole Product right 1 bit. Add and shift happen in parallel → **one cycle per partial-product addition**.
- 32 repetitions; the product is in the 64-bit register at the end.

```
Start → Test Product0 ──1──→ 1a. Left half of Product = left half + Multiplicand ─┐
                      └──0──────────────────────────────────────────────────────┤
        2. Shift the Product register right 1 bit (carry enters MSB) ←──────────┘
        32nd repetition? ── No → back to Test   ── Yes → Done
```

EXAM-READY ANSWER Why the first hardware is inefficient (2019-20 4(b)): it needs a 64-bit ALU and a 64-bit multiplicand register although only 32 bits are ever added, a separate multiplier register, and three sequential steps (add, shift, shift) per bit → about 3 × 32 cycles. Changes: halve the multiplicand register and ALU to 32 bits, shift the product right instead of the multiplicand left, put the multiplier in the unused right half of the product register, and do the add and the shift in the same cycle → 32 cycles, less hardware (draw Fig. 20.2).

4. MIPS Multiplication (sl. 8)

- Two 32-bit registers hold the product: **HI** (most-significant 32 bits) and **LO** (least-significant 32 bits).
- `mult rs, rt` / `multu rs, rt` → 64-bit product in HI/LO (read with mfhi, mflo — textbook).
- `mul rd, rs, rt` → least-significant 32 bits of the product into rd.

5. Combined Multiply/Divide Hardware (textbook, for 2021-22 6(c))

The refined multiplier and the refined divider (Lesson 23) have the same datapath, so one unit does both: a 32-bit **Multiplicand/Divisor** register, a 32-bit **ALU** (add for multiply, subtract for divide), a 64-bit **Product/Remainder** register that can shift **right** (multiply) and **left** (divide) and is written by the ALU into its left half, and a **control** unit that tests Product0 (multiply) or the sign of the ALU result (divide). MIPS exposes the 64-bit register as HI and LO: after mult HI:LO = product; after div LO = quotient, HI = remainder.

Exam Approach

1. Draw both diagrams with register widths (64/32) and shift directions.
2. Flowchart: test, add, shift(s), count 32 (or n) repetitions.
3. Trace table columns: Iteration | Step | Multiplier | Multiplicand | Product (first version) or Multiplicand | Product (refined). Use n-bit operands (n = 4 in exams).

Practice — Lesson 20

(a) Actual — 2023-24 7(a). (b) Actual — 2016-17 8(c). (c) Methods — 2022-23 Q4, 2018-19 5(a), 2014-15 8(b), 2013-14 3(c), 2011-12 6(a), 2021-22 6(c).

(a) Draw the integer multiplication hardware (i) long-multiplication approach and (ii) parallel add/shift approach for 32-bit operands. Simulate each for 1000 (multiplicand) × 1011 (multiplier). (20)

(b) Using the refined hardware (32-bit multiplicand and ALU, 64-bit product shifting right), simulate unsigned 0110 × 0110. (15)

(c) Draw hardware + flowchart of the sequential version and of the refined version; for 2013-14 3(c) 'no separate multiplier register' = the refined version.

✓ Solution

(a) Diagrams: Fig. 20.1 (first) and Fig. 20.2 (refined). 4-bit traces (8 × 11 = 88 = 0101 1000):

Iter	Step (first version)	Multiplier	Multiplicand	Product
0	Initial values	1011	00001000	00000000
1	1a: Multiplier0 = 1 → Prod = Prod + Mcand	1011	00001000	00001000
	2: Shift left Multiplicand	1011	00010000	00001000
	3: Shift right Multiplier	0101	00010000	00001000
2	1a: Multiplier0 = 1 → Prod = Prod + Mcand	0101	00010000	00011000
	2: Shift left Multiplicand	0101	00100000	00011000
	3: Shift right Multiplier	0010	00100000	00011000
3	1: Multiplier0 = 0 → no add	0010	00100000	00011000
	2: Shift left Multiplicand	0010	01000000	00011000
	3: Shift right Multiplier	0001	01000000	00011000
4	1a: Multiplier0 = 1 → Prod = Prod + Mcand	0001	01000000	01011000
	2: Shift left Multiplicand	0001	10000000	01011000
	3: Shift right Multiplier	0000	10000000	01011000

Iter	Step (refined version)	Multiplicand	Product (left | right = multiplier)
0	Initial: Product = 0000 | multiplier	1000	0000 1011
1	1a: Prod0 = 1 → left half += Mcand	1000	1000 1011
	2: Shift right Product (carry enters MSB)	1000	0100 0101
2	1a: Prod0 = 1 → left half += Mcand	1000	1100 0101
	2: Shift right Product (carry enters MSB)	1000	0110 0010
3	1: Prod0 = 0 → no add	1000	0110 0010
	2: Shift right Product (carry enters MSB)	1000	0011 0001
4	1a: Prod0 = 1 → left half += Mcand	1000	1011 0001
	2: Shift right Product (carry enters MSB)	1000	0101 1000

Product = 0101 1000 = 88 = 8 × 11 ✓

(b)

Iter	Step	Multiplicand	Product
0	Initial: Product = 0000 | multiplier	0110	0000 0110
1	1: Prod0 = 0 → no add	0110	0000 0110
	2: Shift right Product (carry enters MSB)	0110	0000 0011
2	1a: Prod0 = 1 → left half += Mcand	0110	0110 0011
	2: Shift right Product (carry enters MSB)	0110	0011 0001
3	1a: Prod0 = 1 → left half += Mcand	0110	1001 0001
	2: Shift right Product (carry enters MSB)	0110	0100 1000
4	1: Prod0 = 0 → no add	0110	0100 1000
	2: Shift right Product (carry enters MSB)	0110	0010 0100

Product = 0010 0100 = 36 = 6 × 6 ✓

(c) §2 and §3 diagrams and flowcharts; the improvement list in the EXAM-READY box; for the combined unit §5.

Lesson 21 — IEEE 754 Floating-Point Representation

Slides 9–16 (§3.5)

1. Floating Point and the Standard (sl. 9–10)

Floating point represents non-integral numbers, including very small and very large ones, like scientific notation: -2.34×10^{56} and $+987.02 \times 10^{9}$… **Normalized** means exactly one non-zero digit before the point (-2.34×10^{56} yes; $+0.002 \times 10^{-4}$ and $+987.02 \times 10^{9}$ no). In binary: $\pm 1.xxxxxxx_2 \times 2^{yyyy}$ (float and double in C). IEEE Std 754-1985 was developed because representations diverged (portability of scientific code); it is now almost universally adopted, with single (32-bit) and double (64-bit) precision.

2. Format (sl. 11) ★

```
single:  | S 1 | Exponent 8  | Fraction 23 |
double:  | S 1 | Exponent 11 | Fraction 52 |
```

$$x = (-1)^S \times (1 + \text{Fraction}) \times 2^{(\text{Exponent} - \text{Bias})}$$

Bias: single 127, double 1023

- S = 0 non-negative, 1 negative.
- Significand normalized: 1.0 ≤ |significand| < 2.0. The leading 1 is always there, so it is not stored (**hidden bit**); significand = fraction with '1.' restored.
- Exponent in excess (biased) form: stored = actual + bias, so the field is unsigned.

EXAM-READY ANSWER Why a (positive) bias (2023-24 7(b), 2018-19 7(c), 2015-16 7(c), 2012-13 8(a), 2020-21 13(a)(i)): with a bias the exponent field is an **unsigned** number — most negative exponent 00…01, most positive 11…10. Then two FP numbers of the same sign can be compared (and sorted) with **integer comparison hardware**, bit pattern by bit pattern, because a larger exponent always gives a larger field. 2's-complement exponents would make negative exponents look larger. The bias $2^{e-1} - 1$ makes the range of exponents roughly symmetric.

3. Ranges and Precision (sl. 12–14)

	Single	Double
Reserved exponent fields	00000000 and 11111111	000…00 and 111…11
Smallest normalized	exp field 1 → 1 − 127 = −126; $\pm 1.0 \times 2^{-126} \approx \pm 1.2 \times 10^{-38}$	exp 1 → −1022; $\pm 1.0 \times 2^{-1022} \approx \pm 2.2 \times 10^{-308}$
Largest	exp 254 → +127; $\pm(2 - 2^{-23}) \times 2^{127} \approx \pm 3.4 \times 10^{38}$	exp 2046 → +1023; $\pm(2 - 2^{-52}) \times 2^{1023} \approx \pm 1.8 \times 10^{308}$
Relative precision	2^{-23} → 23 × $\log_{10}2 \approx$ 6 decimal digits	2^{-52} → ≈ 16 decimal digits
Smallest denormal (textbook)	$2^{-23} \times 2^{-126} = 2^{-149} \approx 1.4 \times 10^{-45}$	$2^{-1074} \approx 4.9 \times 10^{-324}$

4. Special Values (textbook — needed for infinity, NaN and denormal questions)

Exponent field	Fraction	Meaning
0	0	±0
0	≠ 0	Denormal: ±0.Fraction × $2^{1 - \text{bias}}$ (gradual underflow)

Exponent field	Fraction	Meaning
1 … max − 1	anything	Normalized ±1.Fraction × $2^{E - bias}$
all 1s	0	±infinity: single 0 11111111 000…0 = 0x7F800000 (−∞ = 0xFF800000)
all 1s	≠ 0	NaN (not a number), e.g. 0/0, ∞ − ∞

5. Worked Examples (sl. 15–16)

```
Represent -0.75:  -0.75 = (-1)^1 × 1.1_2 × 2^-1
   S = 1, Fraction = 1000…00, Exponent = -1 + 127 = 126 = 01111110 (double: -1 + 1023 =
01111111110)
   single: 1 01111110 1000…00 = 0xBF400000      double: 1 01111111110 1000…00
Decode 1 10000001 01000…00:  S = 1, Fraction = 0.01_2 = 0.25, Exponent = 129
   x = (-1) × (1 + 0.25) × 2^(129 - 127) = -1.25 × 4 = -5.0
```

6. Recipe for a Custom Format (16-bit, 32-bit with other field sizes) ★

1. e = exponent bits, f = fraction bits, 1 + e + f = total. Bias = $2^{e-1} - 1$ (unless the question fixes the range).
2. Normalized exponents: 1 − bias … (2^e − 2) − bias (fields 00…0 and 11…1 reserved).
3. Smallest normalized = 1.0 × $2^{1 - bias}$; largest = (2 − 2^{-f}) × $2^{2^{e} - 2 - bias}$.
4. Smallest denormal = 2^{-f} × $2^{1 - bias}$. Precision = 2^{-f} ≈ f × 0.301 decimal digits.
5. Encode: normalize to 1.xxx × 2^k; field = k + bias in e bits; fraction = bits after the point, padded to f bits.

Exam Approach

1. Write the format box first (S | E | F with widths).
2. Show normalization, biased exponent, fraction padding, then group in 4s → hex.
3. Decode: split fields, check for special exponents, then $(-1)^S(1 + F)2^{E - bias}$.

Practice — Lesson 21

(a) Actual — 2019-20 4(a). (b) Actual — 2022-23 (CSE 305) 8(b) and 2020-21 13(a). (c) Actual — 2018-19 5(b). (d) Actual — 2018-19 5(c), 2017-18 5(c), 2013-14 3(b). (e) Actual — 2023-24 7(b). (f) Actual — 2021-22 8(b). (g) Actual — 2022-23 (CSE 305) 8(c).

(a) 16-bit FP; largest absolute value 1.11…1 × 2^{63}, smallest 1.00…0 × 2^{-62}. (i) Field sizes. (ii) Represent $-110.11_2 \times 2^1$. (iii) Smallest denormalized value. (15)

(b) 32-bit FP with 1 sign bit and (i) 19 fraction bits, (ii) 24 fraction bits, the rest exponent: precision, bias, smallest and largest absolute values, smallest denormal. (8 / 16)

(c) 16-bit FP with a 4-bit exponent: represent −1.25 and 0.375. (8)

(d) Decimal value of IEEE single 0xDF807500 and of 0x7F800005; single representation of −0.75. (5 / 5 / 10)

(e) Purpose of a positive bias; smallest and largest values in single and double precision. (15)

(f) Largest n such that every integer 1, 2, …, n is representable in IEEE single precision. (14)

(g) f1 < f2 are normalized IEEE singles. Treated as unsigned integers i1, i2 — is i1 < i2? As signed integers j1, j2 — is j1 < j2? (20)

✓ Solution

(a) Exponents −62 … +63 = 126 normalized values + 2 reserved = 128 = 2^7 → **7 exponent bits**, bias 63 (field 1 → −62, field 126 → +63); fraction = 16 − 1 − 7 = **8 bits** (the largest has eight 1s after the point).

```
(i)   | S 1 | E 7 (bias 63) | F 8 |
(ii)  −110.11_2 × 2^1 = −1.1011_2 × 2^3 → S = 1, E = 3 + 63 = 66 = 1000010, F = 10110000
      1 1000010 10110000 = 1100 0010 1011 0000 = 0xC2B0   (value −13.5)
(iii) smallest denormal = 0.00000001_2 × 2^(1 − 63) = 2^−8 × 2^−62 = 2^−70 ≈ 8.5 × 10^−22
```

(b)

	(i) 19-bit fraction (2022-23)	(ii) 24-bit fraction (2020-21)
Exponent bits, bias	32 − 1 − 19 = 12 → bias $2^{11} − 1 = 2047$	32 − 1 − 24 = 7 → bias $2^6 − 1 = 63$
Precision	$2^{-19} \approx 1.9 \times 10^{-6}$ (≈ 5.7 decimal digits)	$2^{-24} \approx 6 \times 10^{-8}$ (≈ 7.2 digits)
Smallest normalized	$1.0 \times 2^{1 - 2047} = 2^{-2046}$	1.0×2^{-62}
Largest	$(2 − 2^{-19}) \times 2^{4094 - 2047} = (2 − 2^{-19}) \times 2^{2047}$	$(2 − 2^{-24}) \times 2^{63} \approx 1.84 \times 10^{19}$
Smallest denormal	$2^{-19} \times 2^{-2046} = 2^{-2065}$	$2^{-24} \times 2^{-62} = 2^{-86}$

(c) 4-bit exponent → bias 7, fraction 16 − 1 − 4 = 11 bits.

```
−1.25 = −1.01_2 × 2^0  → S 1, E 0 + 7 = 0111, F 01000000000 → 1 0111 01000000000 = 0xBA00
 0.375 = 1.1_2 × 2^−2   → S 0, E −2 + 7 = 0101, F 10000000000 → 0 0101 10000000000 = 0x2C00
```

(d)

```
0xDF807500 = 1101 1111 1000 0000 0111 0101 0000 0000
  S = 1, E = 1011 1111 = 191 → 191 − 127 = 64, F = 000 0000 0111 0101 0000 0000 = 0x007500
  x = −(1 + 0x7500 / 2^23) × 2^64 = −1.0035705566… × 2^64 ≈ −1.8513 × 10^19
0x7F800005 = 0 11111111 000 0000 0000 0000 0000 0101 → exponent all 1s, fraction ≠ 0 → NaN
−0.75 (2013-14 3(b)) = 1 01111110 1000…0 = 0xBF400000 (slide 15)
```

(e) Bias: EXAM-READY box in §2. Values: table in §3 — single smallest normalized $\pm 2^{-126} \approx \pm 1.18 \times 10^{-38}$, largest $\pm(2 − 2^{-23}) \times 2^{127} \approx \pm 3.40 \times 10^{38}$; double $\pm 2^{-1022} \approx \pm 2.23 \times 10^{-308}$ and $\pm(2 − 2^{-52}) \times 2^{1023} \approx \pm 1.80 \times 10^{308}$ (denormals go down to 2^{-149} and 2^{-1074}).

(f) An integer k is exact when its binary form fits in the 24-bit significand (1 hidden + 23 fraction bits). Every integer up to $2^{24} = 16{,}777{,}216$ fits (2^{24} itself is 1.0×2^{24}). $2^{24} + 1 = 1.000…0001 \times 2^{24}$ needs 25 significant bits → it rounds to 2^{24}. So **n = 2^{24} = 16,777,216** (checked numerically).

(g)

Case	Unsigned i1 < i2?	Signed j1 < j2?
0 ≤ f1 < f2	Yes — sign 0, biased exponent then fraction order = integer order	Yes — both non-negative integers, same order
f1 < 0 ≤ f2	No — i1 has MSB 1, so i1 > i2	Yes — j1 is negative, j2 non-negative
f1 < f2 < 0	No — sign-magnitude: |f1| > |f2| makes i1 > i2	No — larger magnitude bits give the larger (less negative) j: j1 > j2

So neither conclusion holds in general. (i) holds only when both are non-negative; (ii) fails when both are negative. Example: f1 = −3.0 (0xC0400000), f2 = −1.0 (0xBF800000): unsigned 0xC0400000 > 0xBF800000 and signed −1,069,547,520 > −1,082,130,432.

Lesson 22 — Floating-Point Addition and the FP Adder

Slides 17–20 (§3.5; printed 19–22)

1. Decimal Example (sl. 17)

```
9.999 × 10^1 + 1.610 × 10^-1   (4 significant digits)
1. Align decimal points: shift the number with the smaller exponent → 0.016 × 10^1
2. Add significands:     9.999 × 10^1 + 0.016 × 10^1 = 10.015 × 10^1
3. Normalize & check over/underflow: 1.0015 × 10^2
4. Round and renormalize if necessary: 1.002 × 10^2
```

2. Binary Example (sl. 18)

```
1.000_2 × 2^-1 + -1.110_2 × 2^-2     (0.5 + -0.4375)
1. Align: -1.110 × 2^-2 → -0.111 × 2^-1
2. Add:   1.000 × 2^-1 + -0.111 × 2^-1 = 0.001 × 2^-1
3. Normalize: 1.000 × 2^-4, with no over/underflow (-126 ≤ -4 ≤ 127)
4. Round: 1.000 × 2^-4 (no change) = 0.0625
```

3. Flowchart and Exception Conditions ★

```
Start
1. Compare exponents; shift the significand of the smaller number right until the exponents
match
2. Add the significands (with signs)
3. Normalize: shift the sum right & increment exponent, or shift left & decrement exponent
   → Overflow or underflow?  ── Yes → exception (∞ / denormal or 0)
4. Round the significand to the available bits
   → Still normalized? ── No → back to step 3
Done
```

Condition	When (single precision)
Overflow	after normalizing, the exponent is > 127 (field would be 255) → result ±∞
Underflow	after normalizing, the exponent is < −126 → denormal or 0
Infinity result	an operand is ±∞ (∞ + x = ∞), or overflow
NaN result	an operand is NaN, or (+∞) + (−∞)

4. FP Adder Hardware (sl. 19–20)

The FP adder is much more complex than an integer adder. Doing it in one clock cycle would take too long (much longer than integer operations) and a slower clock would penalize all instructions, so the FP adder usually takes several cycles and can be pipelined.

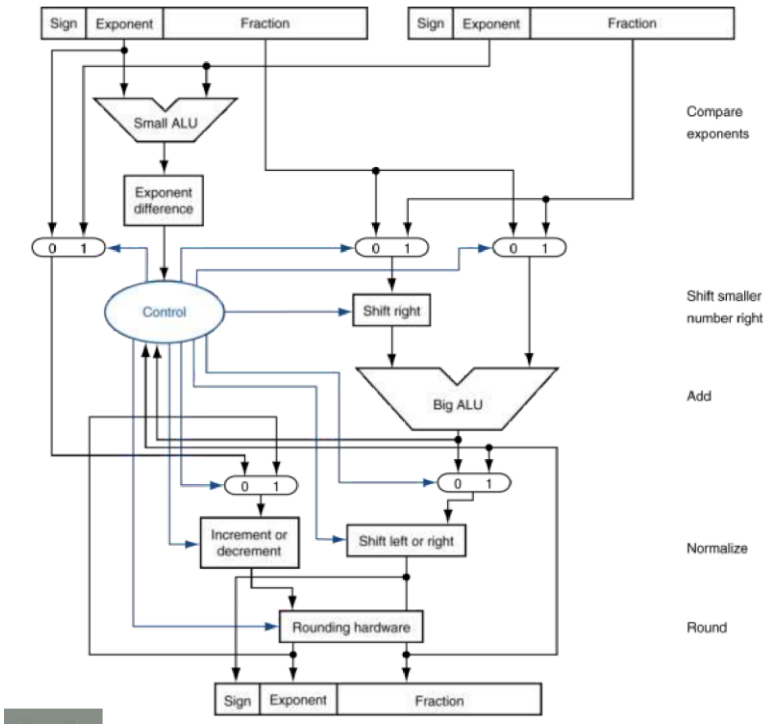


Figure 22.1 — FP adder: small ALU (exponent difference), shift right, big ALU, normalize, round (sl. 20)

- Step 1: the **small ALU** subtracts the exponents; the difference controls the muxes and the **shift-right** unit that aligns the smaller significand.
- Step 2: the **big ALU** adds the significands.
- Step 3: **normalize** — the leading-zero logic shifts left or right and the **increment/decrement** unit adjusts the exponent.
- Step 4: **rounding hardware**; if rounding de-normalizes the result, go back to step 3.

Exam Approach

1. Convert both numbers to normalized binary (enough fraction bits).
2. Align the smaller exponent to the larger one by shifting its significand right.
3. Add with signs, normalize, check the exponent range, round to the format's fraction bits.
4. Give the final bit pattern and its decimal value as a check.

Practice — Lesson 22

(a) Actual — 2016-17 7(b). (b) Actual — 2017-18 5(b). (c) Actual — 2022-23 1(a), 2018-19 8(b), 2015-16 6(a), 2013-14 3(a), 2011-12 5(a).

(a) Using the FP adder hardware, simulate −(0.25) + (0.625) by filling: binary, step 1 exponent matching, step 2 significand addition, step 3 normalization, step 4 rounding. (15)

(b) 16-bit half precision with a 4-bit exponent: convert 0.75 and 1.25 and add them step by step, giving the exponent and fraction fields at every step. (15)

(c) Steps of FP addition (flowchart) with an example; overflow, underflow, NaN and infinity conditions. (10–12)

✓ Solution

(a)

	$-(0.25)_{10}$	$(0.625)_{10}$
Binary	-1.000×2^{-2} (1 01111101 000…0)	1.010×2^{-1} (0 01111110 010…0)

	$-(0.25)_{10}$	$(0.625)_{10}$
Step 1: exponent matching	-0.100×2^{-1} (shift right 1, exp $-2 \rightarrow -1$)	1.010×2^{-1} (unchanged)
Step 2: significand addition	$1.010 + (-0.100) = 0.110 \times 2^{-1}$	
Step 3: normalization	$0.110 \times 2^{-1} = 1.100 \times 2^{-2}$; $-126 \le -2 \le 127 \rightarrow$ no over/underflow	
Step 4: rounding	1.100×2^{-2} (no change) = 0 01111101 1000…0 = 0x3EC00000 = 0.375	

(b) Bias 7, 11 fraction bits.

```
0.75 = 1.1_2 × 2^−1  → S 0, E 0110 (6), F 10000000000   = 0x3400
1.25 = 1.01_2 × 2^0  → S 0, E 0111 (7), F 01000000000   = 0x3A00
Step 1 (align): exponents 6 < 7 → shift 0.75: 1.10000000000 × 2^−1 → 0.11000000000 × 2^0, E =
0111
Step 2 (add):   1.01000000000 + 0.11000000000 = 10.00000000000 × 2^0
Step 3 (normalize): 1.00000000000 × 2^1 → E = 1 + 7 = 1000; 1 ≤ 8 ≤ 14 → no overflow/underflow
Step 4 (round): no bits were lost → F = 00000000000
Result: 0 1000 00000000000 = 0x4000 = 2.0 ✓
```

(c) Flowchart and condition table in §3 with the slide-18 binary example (0.5 + −0.4375 = 0.0625).

Lesson 23 — Beyond the Short Slides (textbook insurance)

Not on Tushar Sir's short slides — P&H §3.3–3.6. Asked: division ×4, MIPS FP code ×4, signed multiplication, guard/round/sticky, subword parallelism

> **NOTE** These questions are in the tracker's Outside Notes tab because the short slides do not cover them. Division and FP code were asked as recently as 2022-23 (CSE 209 1(b), CSE 305 8(a)) — ask sir whether they are in this term's syllabus.

1. Division Hardware and Algorithm (§3.4)

First version: 64-bit **Divisor** register (divisor starts in the LEFT half) shifting right, 64-bit ALU, 64-bit **Remainder** register (initially the dividend), 32-bit **Quotient** shifting left. 33 repetitions:

```
1. Remainder = Remainder – Divisor
2. Test Remainder:  ≥ 0 → shift Quotient left, set new rightmost bit 1
                    < 0 → restore: Remainder = Remainder + Divisor; shift Quotient left, new
bit 0
3. Shift Divisor right 1 bit;   33rd repetition? No → step 1, Yes → Done
```

Refined version (the hardware of 2012-13 8(b)): 32-bit Divisor, **32-bit ALU**, 64-bit Remainder register that shifts left; the quotient bits enter its right half. Algorithm: shift Remainder left 1; repeat n times {left half − Divisor; if ≥ 0 keep it and shift left with new bit 1, else restore and shift left with new bit 0}; finally shift the left half right 1 bit. Then left half = remainder, right half = quotient. It needs the dividend's upper half < divisor (otherwise the quotient overflows n bits).

2022-23 1(b) ★ — 6-bit ALU: 1011 1100 1111 ÷ 110101 (3023 ÷ 53)

Iter	Step (refined, 6-bit ALU)	Divisor	Remainder register (left | right)
0	Initial: Rem = dividend	110101	101111 | 001111
	Shift Rem left 1	110101	1011110 | 011110
1	Left half − Div ≥ 0 → keep; shift Rem left, R0 = 1	110101	1010010 | 111101
2	Left half − Div ≥ 0 → keep; shift Rem left, R0 = 1	110101	0111011 | 111011
3	Left half − Div ≥ 0 → keep; shift Rem left, R0 = 1	110101	0001101 | 110111
4	Left half − Div < 0 → restore; shift Rem left, R0 = 0	110101	0011011 | 101110
5	Left half − Div < 0 → restore; shift Rem left, R0 = 0	110101	0110111 | 011100
6	Left half − Div ≥ 0 → keep; shift Rem left, R0 = 1	110101	0000100 | 111001
	Shift left half right 1 (final fix-up)	110101	000010 111001

Quotient = 111001 = 57, Remainder = 000010 = 2 (57 × 53 + 2 = 3023 ✓)

2018-19 6(b) — 4-bit: 11001001 ÷ 1011 (201 ÷ 11), first version

Iter	Step	Quotient	Divisor	Remainder
0	Initial values	00000	10110000	11001001
1	1: Rem = Rem − Div → Rem ≥ 0	00000	10110000	00011001
	2a: Shift Q left, Q0 = 1	00001	10110000	00011001
	3: Shift Divisor right	00001	01011000	00011001
2	1: Rem = Rem − Div → Rem < 0	00001	01011000	11000001

Iter	Step	Quotient	Divisor	Remainder
	2b: Rem = Rem + Div (restore); shift Q left, Q0 = 0	00010	01011000	00011001
	3: Shift Divisor right	00010	00101100	00011001
3	1: Rem = Rem − Div → Rem < 0	00010	00101100	11101101
	2b: Rem = Rem + Div (restore); shift Q left, Q0 = 0	00100	00101100	00011001
	3: Shift Divisor right	00100	00010110	00011001
4	1: Rem = Rem − Div → Rem ≥ 0	00100	00010110	00000011
	2a: Shift Q left, Q0 = 1	01001	00010110	00000011
	3: Shift Divisor right	01001	00001011	00000011
5	1: Rem = Rem − Div → Rem < 0	01001	00001011	11111000
	2b: Rem = Rem + Div (restore); shift Q left, Q0 = 0	10010	00001011	00000011
	3: Shift Divisor right	10010	00000101	00000011

Quotient = 10010 = 18, Remainder = 0011 = 3 (18 × 11 + 3 = 201 ✓)

Note: the dividend's upper half 1100 ≥ divisor 1011, so the quotient needs 5 bits — with the refined 4-bit hardware this is a divide overflow; the first version (5 iterations) handles it.

2017-18 8(a) — 11010101 ÷ 1011 (213 ÷ 11), first version

Iter	Step	Quotient	Divisor	Remainder
0	Initial values	00000	10110000	11010101
1	1: Rem = Rem − Div → Rem ≥ 0	00000	10110000	00100101
	2a: Shift Q left, Q0 = 1	00001	10110000	00100101
	3: Shift Divisor right	00001	01011000	00100101
2	1: Rem = Rem − Div → Rem < 0	00001	01011000	11001101
	2b: Rem = Rem + Div (restore); shift Q left, Q0 = 0	00010	01011000	00100101
	3: Shift Divisor right	00010	00101100	00100101
3	1: Rem = Rem − Div → Rem < 0	00010	00101100	11111001
	2b: Rem = Rem + Div (restore); shift Q left, Q0 = 0	00100	00101100	00100101
	3: Shift Divisor right	00100	00010110	00100101
4	1: Rem = Rem − Div → Rem ≥ 0	00100	00010110	00001111
	2a: Shift Q left, Q0 = 1	01001	00010110	00001111
	3: Shift Divisor right	01001	00001011	00001111
5	1: Rem = Rem − Div → Rem ≥ 0	01001	00001011	00000100
	2a: Shift Q left, Q0 = 1	10011	00001011	00000100
	3: Shift Divisor right	10011	00000101	00000100

Quotient = 10011 = 19, Remainder = 0100 = 4 (19 × 11 + 4 = 213 ✓)

2. Signed Multiplication (2020-21 10(c))

Simplest method: remember the signs, convert both operands to positive (2's-complement negate the negative ones), multiply the magnitudes with the unsigned algorithm for 31 iterations, and negate the product if the original signs differed (sign = $S_A \oplus S_B$). Alternatively, sign-extend the partial products and

treat the last step as a subtraction (Booth's algorithm: look at multiplier bits (b_i, b_{i-1}): 10 → subtract multiplicand, 01 → add, 00/11 → only shift, using arithmetic right shifts) — this works directly on 2's-complement numbers.

3. MIPS Floating-Point Instructions and Code (§3.5)

- Separate FP registers $f0–$f31; a double uses an even/odd pair ($f0/$f1 …).
- Arithmetic: add.s / add.d, sub.s / sub.d, mul.s / mul.d, div.s / div.d. Loads/stores: lwc1 / swc1 (single), ldc1 / sdc1 (double).
- Compare: c.xx.s / c.xx.d (xx = eq, lt, le) sets a condition bit; branch with bc1t (true) / bc1f (false).

2022-23 (CSE 305) 8(a) = 2020-21 13(c) ★: float a, b, c; double w, x, y, z; if (a + b > c) x = (y*z)/w;

```
# a $f0, b $f1, c $f2 (single);  w $f4, x $f6, y $f8, z $f10 (double pairs)
      add.s  $f3, $f0, $f1      # f3 = a + b
      c.le.s $f3, $f2           # (a + b) <= c ?
      bc1t   Skip               # yes → condition false, skip
      mul.d  $f12, $f8, $f10    # y * z
      div.d  $f6, $f12, $f4     # x = (y * z) / w
Skip:
```

2019-20 4(c) ★: double; if (a > b) c = c − (c*e)/d; else c = c + d; (a $f2, b $f4, c $f6, d $f8, e $f10)

```
      c.le.d $f2, $f4           # a <= b ?
      bc1t   Else
      mul.d  $f12, $f6, $f10    # c * e
      div.d  $f12, $f12, $f8    # (c * e) / d
      sub.d  $f6, $f6, $f12     # c = c - (c*e)/d
      j      Exit
Else: add.d  $f6, $f6, $f8      # c = c + d
Exit:
```

2017-18 8(b): float convert(float cels) { return 9.0*cels/5.0 + 32.0; } (cels in $f12, result in $f0)

```
convert: lwc1  $f16, const9($gp)   # 9.0 from static data
         lwc1  $f18, const5($gp)   # 5.0
         mul.s $f16, $f16, $f12    # 9.0 * cels
         div.s $f16, $f16, $f18    # / 5.0
         lwc1  $f18, const32($gp)  # 32.0
         add.s $f0, $f16, $f18     # + 32.0
         jr    $ra
```

4. Guard, Round and Sticky Bits (2020-21 12(c))

During alignment and normalization, bits shifted out of the significand would be lost. IEEE 754 keeps two extra bits on the right during the operation — **guard** and **round** — so that the result can be rounded as if it had been computed exactly and then rounded. Example with 3 significant decimal digits: $2.56 \times 10^0 + 2.34 \times 10^2$: with guard/round digits 2.3400 + 0.0256 = 2.3656 → rounds to 2.37; without them 2.34 + 0.02 = 2.36 (an error of 1 in the last digit). The **sticky bit** is set if any nonzero bit was shifted beyond the round bit; it tells round-to-nearest-even whether the discarded part is exactly half (…100…0) or more than half, so ties are broken correctly. Together they give results accurate to ½ unit in the last place.

5. Subword Parallelism (2021-22 5(c))

A wide (64/128-bit) adder can be partitioned so that one instruction operates on many short operands at once: 16 × 8-bit, 8 × 16-bit, 4 × 32-bit or 2 × 64-bit (SIMD — MMX/SSE/AVX, ARM NEON). Image processing works on 8-bit pixels, so 16 pixels are added, averaged or saturated in one instruction; matrix multiplication uses packed multiply-add on 4 floats/doubles at a time, turning the inner loop into one quarter (or less) of the scalar instructions with the same hardware.

Quick Notes — Arithmetic

Revision points for Lessons 19–23.

L19 · Add/sub overflow

- Add overflow: (+)+(+) → sign 1; (−)+(−) → sign 0. Subtract: (+)−(−) → 1; (−)−(+) → 0. Never for opposite-sign add / same-sign subtract.
- $V = O'S_A'S_B'S_R + O'S_AS_BS_R' + OS_A'S_BS_R + OS_AS_B'S_R'$.

L20 · Multiplication

- First: 64-bit multiplicand ←, 32-bit multiplier →, 64-bit ALU & product; add, shift, shift × 32.
- Refined: 32-bit multiplicand & ALU, multiplier in right half of 64-bit product, add then shift product → (1 cycle/step).
- HI/LO; mult/multu → HI:LO; mul → low 32 bits. Combined mult/div: same refined datapath, product register shifts both ways.

L21 · IEEE 754

- $x = (-1)^S(1 + F)2^{E - bias}$; single 1/8/23 bias 127; double 1/11/52 bias 1023.
- Single 2^{-126} … $(2 - 2^{-23})2^{127}$; denormal 2^{-149}. ∞: E all 1s, F = 0 (0x7F800000). NaN: E all 1s, F ≠ 0.
- Bias → unsigned exponent → integer comparison works. Custom: bias $2^{e-1} - 1$; min 2^{1-bias}; max $(2 - 2^{-f})2^{2^{e}-2-bias}$; denorm $2^{-f} \cdot 2^{1-bias}$.
- −0.75 = 0xBF400000; 0xC0A00000 = −5.0; 0xDF807500 ≈ -1.85×10^{19}; integers exact up to 2^{24}.

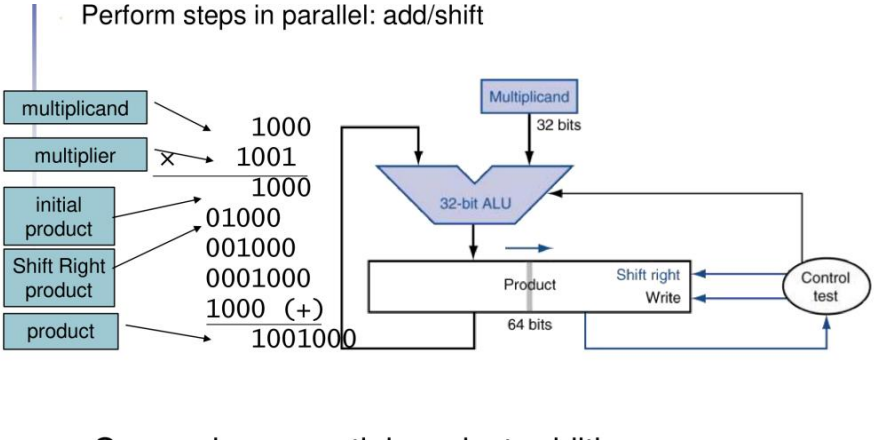
L22 · FP addition

- Align (shift smaller right) → add → normalize + check over/underflow → round (→ renormalize).
- −0.25 + 0.625 = $1.1 \times 2^{-2} = 0.375$; half precision 0.75 + 1.25 = 0x4000 = 2.0.

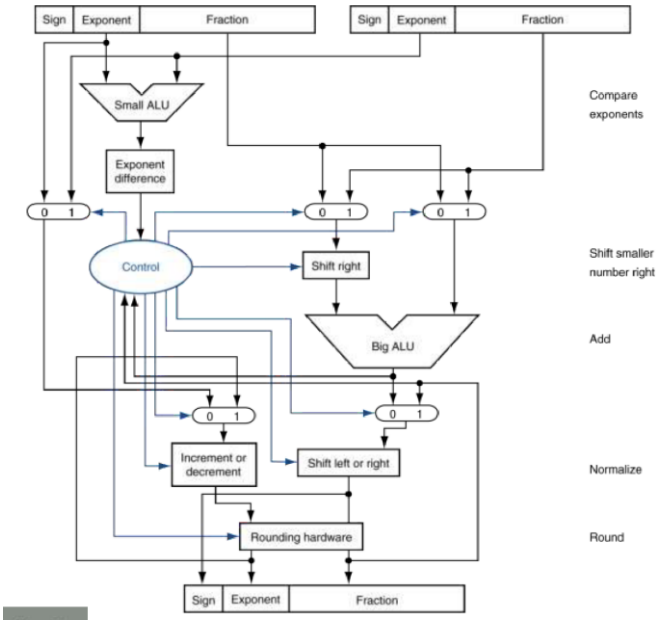
L23 · Textbook extras

- Division: subtract, test sign, restore or set 1, shift; refined: remainder register shifts left, quotient enters right half; final left-half shift right.
- 3023/53 = 57 r 2; 201/11 = 18 r 3; 213/11 = 19 r 4.
- FP code: add.s/d, mul, div, c.le.s + bc1t; doubles in even registers.
- Guard + round + sticky → correct rounding; subword parallelism = partitioned adder (SIMD).

Key Diagrams to Reproduce



Optimized multiplier (sl. 7) — same as Fig. 20.2



FP adder hardware (sl. 20) — same as Fig. 22.1